Chapter 4
Device Management (DM)

✓the important jobs of an Operating System is to manage various I/O devices .

E.g. mouse, keyboards, touch pad, disk drives, LED, Analog-to-digital converter

- An I/O system is required to take an application I/O request and send it to the physical device,

- then take whatever response comes back from the device and send it to the application.

I/O devices can be divided into two categories:-

1. Block devices

✓A block device is one with which the driver communicates by sending entire blocks of data.

For example, Hard disks, USB cameras, Disk-On-Key etc.

2. Character devices

✓A character device is one with which the driver communicates by sending and receiving single characters (bytes, octets).

For example, serial ports, parallel ports, sounds cards etc.

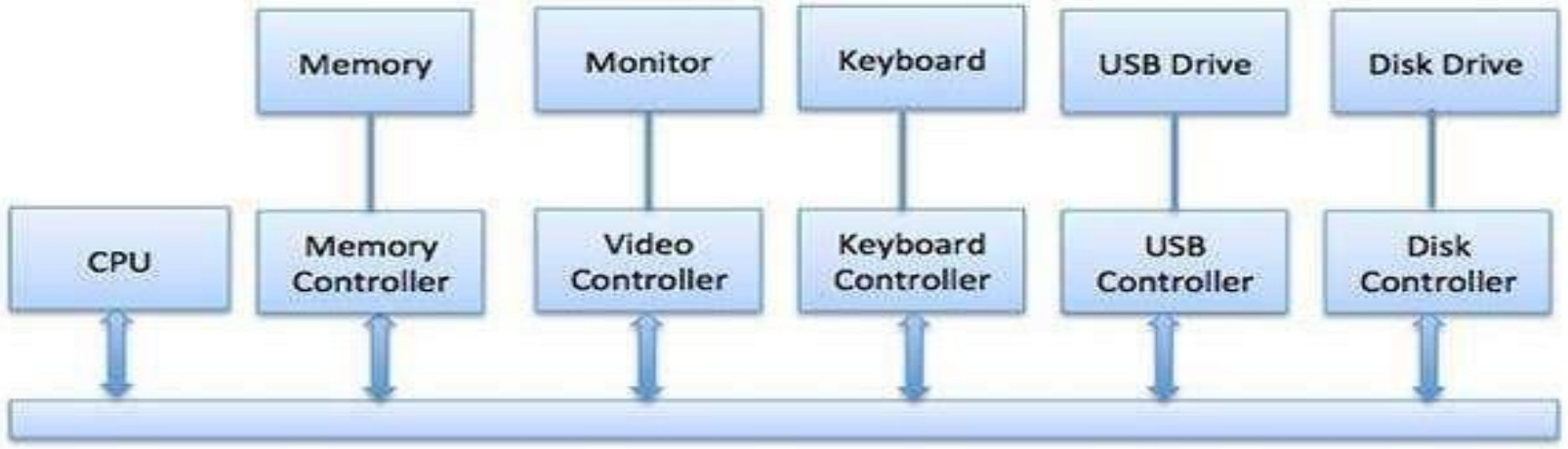
Device Controllers

- *Device drivers* are software modules that can be plugged into an OS to handle a particular device.
- Operating System takes help from device drivers to handle all I/O devices.
- The *Device Controller* works like an interface between a device and a device driver.
- I/O units (Keyboard, mouse, printer, etc.) typically consist of a mechanical component and an electronic component where electronic component is called the device controller.
- There is always a device controller and a device driver for each device to communicate with the Operating Systems.
- A device controller may be able to handle multiple devices.

- As an interface its main task is to convert serial bit stream to block of bytes, perform error correction as necessary.

- Any device connected to the computer is connected by a *plug* and *socket*, and the socket is connected to a *device controller*.

- Following is a model for connecting the CPU, memory, controllers, and I/O devices where CPU and device controllers all use a common bus for communication.



Synchronous vs Asynchronous I/O

Synchronous I/O

✓ In this scheme, CPU execution waits while I/O proceeds.

Asynchronous I/O

✓I/O proceeds concurrently with CPU execution.

Communication to I/O Devices

✓The CPU must have a way to pass information to and from an I/O device.

✓There are three approaches available to communicate with the CPU and Device.

1. Special Instruction I/O

2. Memory-mapped I/O

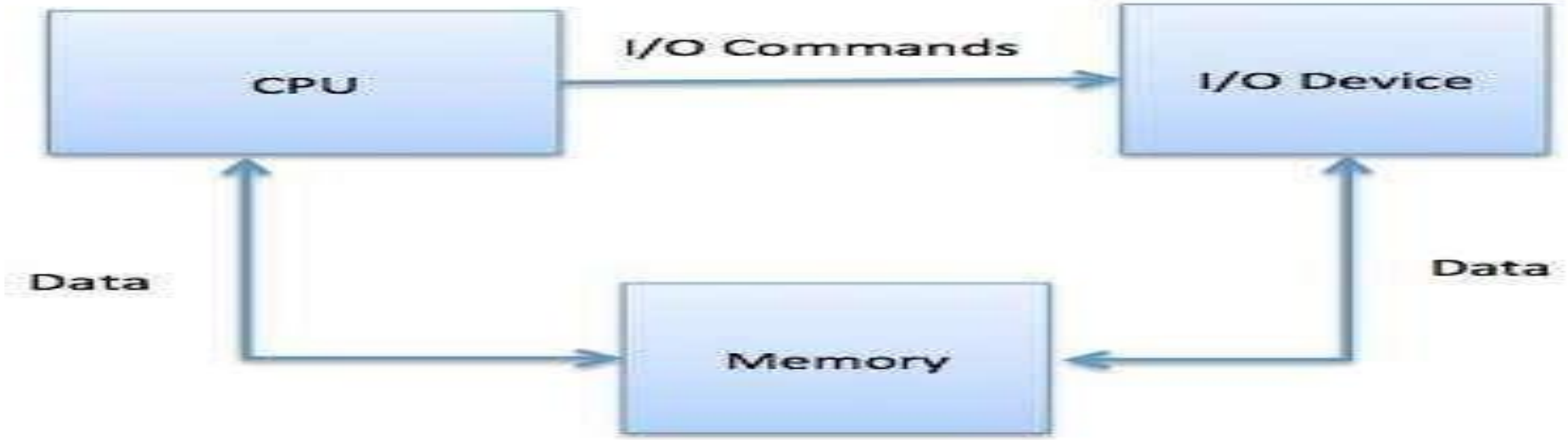
3. Direct memory access (DMA)

Special Instruction I/O

- ✓This uses CPU instructions that are specifically made for controlling I/O devices.
- ✓These instructions typically allow data to be sent to an I/O device or read from an I/O device.

Memory-mapped I/O

- ✓When using memory-mapped I/O, the same address space is shared by memory and I/O devices.
- ✓The device is connected directly to certain main memory locations so that I/O device can transfer block of data to/from memory without going through CPU.



- While using memory mapped IO, OS allocates buffer in memory and informs I/O device to use that buffer to send data to the CPU.

- I/O device operates asynchronously with CPU, interrupts CPU when finished.

- The advantage to this method is that every instruction which can access memory can be used to manipulate an I/O device.

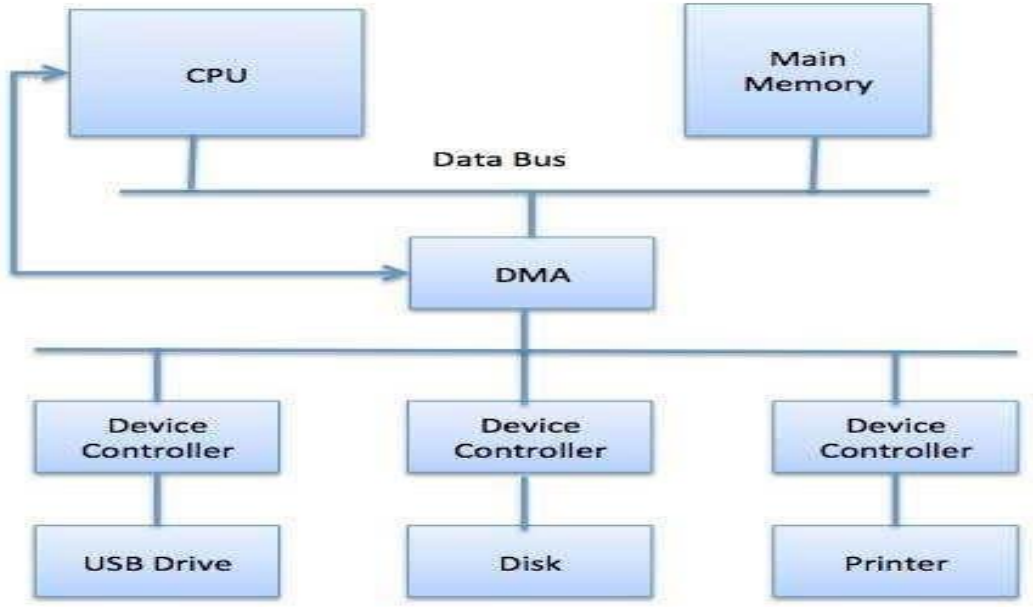
- Memory mapped IO is used for most high-speed I/O devices like disks, communication interfaces.

Direct Memory Access (DMA)

- ✓Slow devices like keyboards will generate an interrupt to the main CPU after each byte is transferred.
- ✓If a fast device such as a disk generated an interrupt for each byte, the operating system would spend most of its time handling these interrupts.
- ✓So, a typical computer uses direct memory access (DMA) hardware to reduce this overhead.
- ✓Direct Memory Access (DMA) means CPU grants I/O module authority to read from or write to memory without involvement.
- ✓DMA module itself controls exchange of data between main memory and the I/O device.

- ✓CPU is only involved at the beginning and end of the transfer and interrupted only after entire block has been transferred.

- ✓Direct Memory Access needs a special hardware called *DMA controller* (DMAC) that manages the data transfers and arbitrates access to the system bus.
- ✓The controllers are programmed with source and destination pointers (where to read/write the data),
- ✓counters to track the number of transferred bytes, and settings, which includes I/O and memory types, interrupts and states for the CPU cycles.



✓The operating system uses the DMA hardware as follows:-

Step	Description
1	Device driver is instructed to transfer disk data to a buffer address X.
2	Device driver then instruct disk controller to transfer data to buffer.
3	Disk controller starts DMA transfer.
4	Disk controller sends each byte to DMA controller.
5	DMA controller transfers bytes to buffer, increases the memory address, decreases the counter C until C becomes zero.
6	When C becomes zero, DMA interrupts CPU to signal transfer completion.

Polling vs Interrupts I/O

- ✓A computer must have a way of detecting the arrival of any type of input.
- ✓There are two ways that this can happen, known as **polling** and **interrupts**.
- ✓Both of these techniques allow the processor to deal with events that can happen at any time and that are not related to the process it is currently running.

Polling I/O

- ✓*Polling* is the simplest way for an I/O device to communicate with the processor to the processor.
- ✓The process of periodically checking status of the device to see if it is time for the next I/O operation, is called *polling*.
- ✓The I/O device simply puts the information in a Status register, and the processor must come and get the information.
- ✓Most of the time, devices will not require attention and when one does it will have to wait until it is next interrogated by the polling program.

- ✓This is an inefficient method and much of the processors time is wasted on unnecessary polls.
- ✓Compare this method to a teacher continually asking every student in a class, one after another, if they need help.
- ✓Obviously the more efficient method would be for a student to inform the teacher whenever they require assistance.

Interrupts I/O

- ✓An alternative scheme for dealing with I/O is the interrupt-driven method.
- ✓ An interrupt is a signal to the microprocessor from a device that requires attention.

- ✓A device controller puts an interrupt signal on the bus when it needs CPU's attention when CPU receives an interrupt,
- ✓it saves its current state and invokes the appropriate interrupt handler using the interrupt vector (addresses of OS routines to handle various events).

✓When the interrupting device has been dealt with, the CPU continues with its original task as if it had never been interrupted.

Software input and output management

✓I/O software is often organized in the following layers:

User Level Libraries:

✓This provides simple interface to the user program to perform input and output.

✓For example, **stdio** is a library provided by C and C++ programming languages.

Kernel Level Modules:

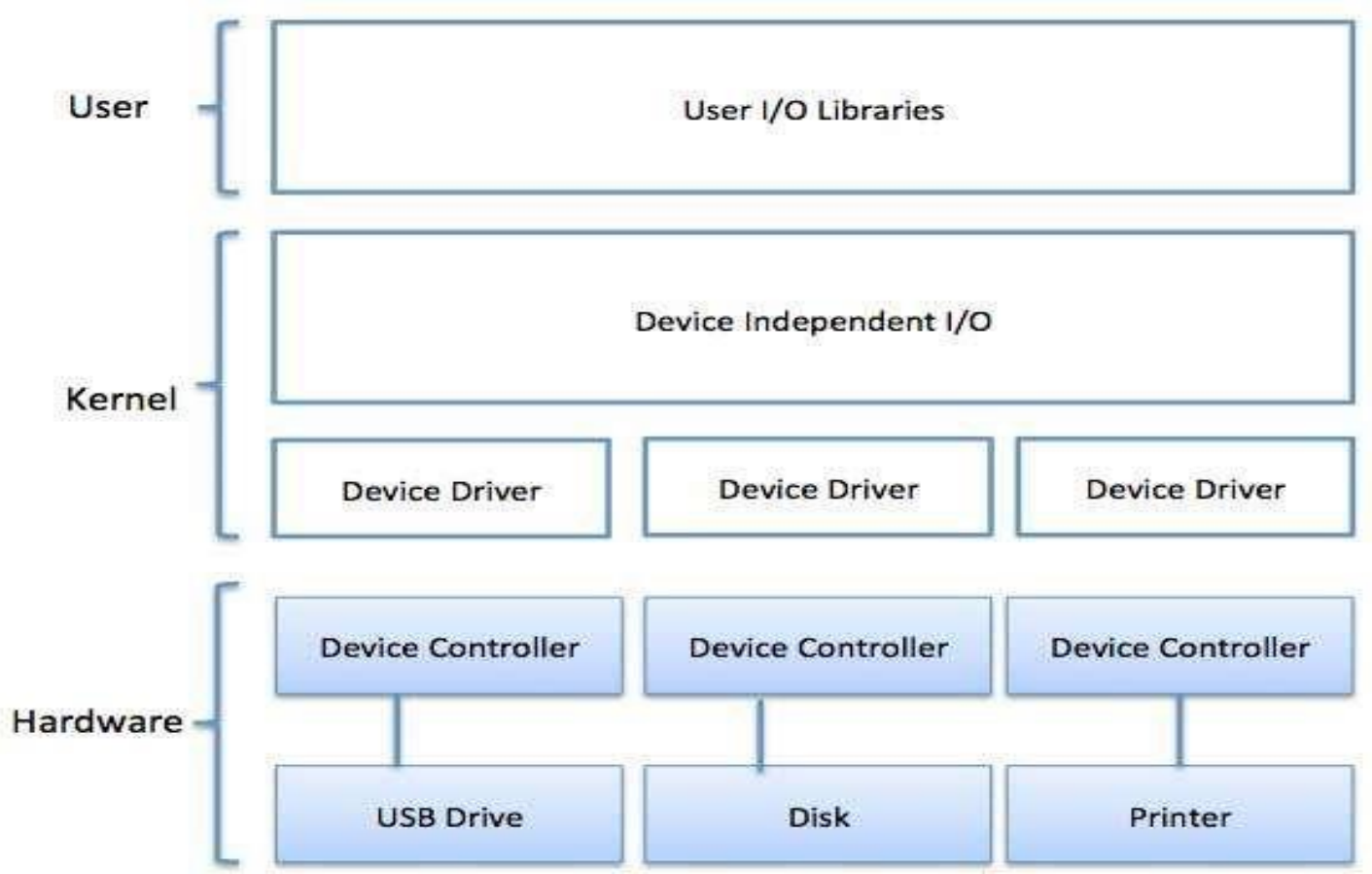
✓This provides device driver to interact with the device controller and device independent I/O modules used by the device drivers.

Hardware:

✓This layer includes actual hardware and hardware controller which interact with the device drivers and makes hardware alive.

✓A key concept in the design of I/O software is that it should be device independent where it should be possible to write programs that can access any I/O device without having to specify the device in advance.

✓ For example, a program that reads a file as input should be able to read a file on a floppy disk, on a hard disk, or on a CD-ROM, without having to modify the program for each different device.



Device drivers

✓is software modules that can be plugged into an OS to handle a particular device.

✓Operating System takes help from device drivers to handle all I/O devices.

- Device drivers encapsulate device-dependent code and implement a standard interface in such a way that code contains **device-specific register reads/writes**.

- Device driver, is generally written by the device's manufacturer and delivered along with the device on a CD-ROM.

A device driver performs the following jobs:-

- To accept request from the device independent software above to it.
- Interact with the device controller to take and give I/O and perform required error handling
- Making sure that the request is executed successfully

How a device driver handles a request is as follows: -

- Suppose a request comes to read a block N.
- If the driver is idle at the time a request arrives, it starts carrying out the request immediately.
- Otherwise, if the driver is already busy with some other request,
- it places the new request in the queue of pending requests.

Interrupt handlers

- ✓An interrupt handler, also known as an *interrupt service routine* or ISR,
- ✓is a piece of software or more specifically a callback function in an operating system or more specifically in a device driver, whose execution is triggered by the reception of an interrupt.
- ✓When the interrupt happens, the interrupt procedure does whatever it has to in order to handle the interrupt, updates data structures and wakes up process that was waiting for an interrupt to happen.
- ✓The interrupt mechanism accepts an address a number that selects a specific interrupt handling routine/function from a small set.

Device-Independent I/O Software

- is to perform the I/O functions that are common to all devices and
- to provide a uniform interface to the user-level software.

- Though it is difficult to write completely device independent software but we can write some modules which are common among all the devices.

Following is a list of functions of device-independent I/O Software:-

- ✓Uniform interfacing for device drivers
- ✓Device naming Mnemonic names mapped to Major and Minor device numbers
- ✓Device protection
- ✓Providing a device-independent block size
- ✓Buffering because data coming off a device cannot be stored in final destination.
- ✓Storage allocation on block devices
- ✓Allocation and releasing dedicated devices
- ✓Error Reporting

User-Space I/O Software

- ✓These are the libraries which provide richer and simplified interface to access the functionality of the kernel or ultimately interactive with the device drivers.

- ✓ Most of the user-level I/O software consists of library procedures with some exception like spooling system

- ✓which is a way of dealing with dedicated I/O devices in a multiprogramming system.
- ✓ I/O Libraries (e.g., stdio) are in user- space to provide an interface to the OS resident device-independent I/O SW.

- ✓For example putchar(), getchar(), printf() and scanf() are example of user level I/O library stdio available in C programming.

putchar() function in C

- The **putchar(int ch)** method in C is used to write a character, of unsigned char type, to stdout.
- This character is passed as the parameter to this method.

Syntax:

```
int putchar(int ch)
```

- **Parameters:** This method accepts a mandatory parameter **ch** which is the character to be written to stdout.
- **Return Value:** This function returns the character written on the stdout as an unsigned char. It also returns EOF when some error occurs.

```c
// C program to demonstrate putchar() method
#include <stdio.h>
int main()
{
      // Get the character to be written
    char ch = 'G';
    // Write the Character to stdout
    putchar(ch);
    return (0);
}
```

```
aman@aman-VirtualBox:~/Desktop$ cc putchar.c
aman@aman-VirtualBox:~/Desktop$ ./a.out
G
aman@aman-VirtualBox:~/Desktop$
```

putchar() function in C

- is a standard library function that takes a single input character from standard input.
- The major difference between getchar and getc is that getc can take input from any no of input streams but

- getchar can take input from a single standard input stream.

- It is defined inside the **<stdio.h> header file.**

- Just like getchar, there is also a function called putchar that prints only one character to the standard output stream.

Syntax :

```
int getchar(void);
```

✓getchar() function does not take any parameters.

putchar() function in C

Return Value

- The input from the standard input is read as an unsigned char and then it is typecast and returned as an integer value(int).

EOF is returned in two cases:

- When the file end is reached
- When there is an error during the execution

```c
// C program to implement getchar()
// function to read single character
#include <stdio.h>

// Driver code
int main()
{
    int character;
    character = getchar();

    printf("The entered character is : %c", character);
  printf("\n");
    return 0;
}
```

```
aman@aman-VirtualBox:~/Desktop$ chmod +x getchar.c
aman@aman-VirtualBox:~/Desktop$ cc getchar.c
aman@aman-VirtualBox:~/Desktop$ ./a.out
A
The entered character is : A
aman@aman-VirtualBox:~/Desktop$
```

Kernel I/O Subsystem

- ✓Kernel I/O Subsystem is responsible to provide many services related to I/O.

Following are some of the services provided:-

Scheduling –

- ✓ Kernel schedules a set of I/O requests to determine a good order in which to execute them.
- ✓ When an application issues a blocking I/O system call, the request is placed on the queue for that device.

- ✓The Kernel I/O scheduler rearranges the order of the queue to improve the overall system efficiency and the average response time experienced by the applications.

Buffering –

- Kernel I/O Subsystem maintains a memory area known as **buffer** that stores data while they are transferred between two devices or between a devices with an application operation.

- Buffering is done to cope with a speed mismatch between the producer and consumer of a data stream or to adapt between devices that have different data transfer sizes.

Caching –

- Kernel maintains cache memory which is region of fast memory that holds copies of data.

- Access to the cached copy is more efficient than access to the original.

Spooling and Device Reservation

- A spool is a buffer that holds output for a device, such as a printer, that cannot accept interleaved data streams.
- The spooling system copies the queued spool files to the printer one at a time.
- In some operating systems, spooling is managed by a system daemon process.
- In other operating systems, it is handled by an in-kernel thread.

***Error Handling* –**

- An operating system that uses protected memory can guard against many kinds of hardware and application errors.

System Recovery

- ✓The recovery process is designed to recover a server to a previous operating state,
- ✓in the event of a hardware or operating system failure.
- ✓The recovery process will begin by starting your computer using a Bootable Recover Assist Media or a Bootable Backup Media.
- ✓The process will then load a recovery environment, which you can use to select the location of the backup to be used, and
- ✓to start the recovery.

Input-output system calls

- System calls are the calls that a program makes to the system kernel to provide the services to which the program does not have direct access.

- For example, providing access to input and output devices such as monitors and keyboards.

- We can use various functions provided in the C Programming language for input/output system calls such as create, open, read, write, etc.

- Before we move on to the I/O System Calls, we need to know about a few important terms.

Important Terminology

What is the File Descriptor?

✓The file descriptor is an integer that uniquely identifies an open file of the process.

File Descriptor table: is the collection of integer array indices that are file descriptors in which elements are pointers to file table entries.

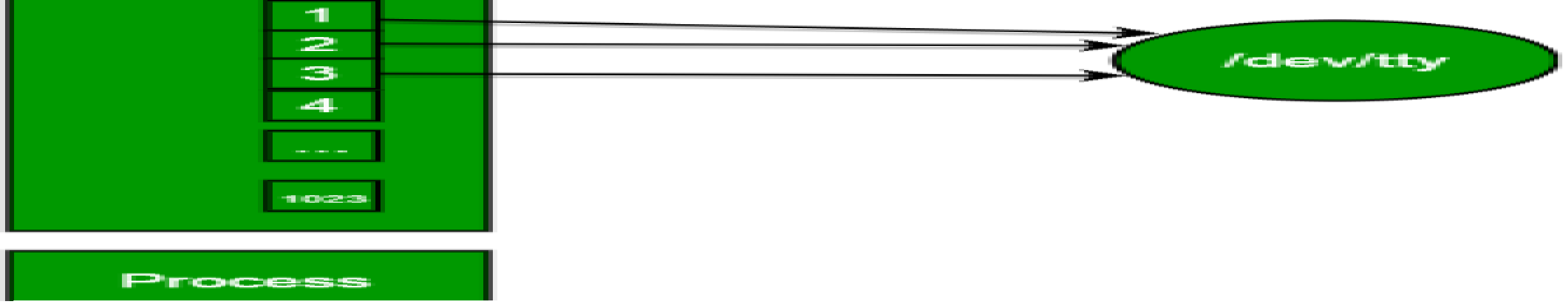
✓One unique file descriptors table is provided in the operating system for each process.

File Table Entry:

File table entries are a structure In-memory surrogate for an open file, which is created when processing a request to open the file and these entries maintain file position.



Standard File Descriptors:
When any process starts, then that process file descriptors table's fd(file descriptor) 0, 1, 2 open automatically,
(By default) each of these 3 fd references file table entry for a file named **/dev/tty**
/dev/tty: In-memory surrogate for the terminal.
Terminal: Combination keyboard/video screen.



Read from stdin => read from fd 0: Whenever we write any character from the keyboard, it reads from stdin through fd 0 and saves to a file named /dev/tty.

Write to stdout => write to fd 1: Whenever we see any output to the video screen, it's from the file named /dev/tty and written to stdout in screen through fd 1.

Write to stderr => write to fd 2: We see any error to the video screen, it is also from that file write to stderr in screen through fd 2.

Input/output System Calls

Basically, there are total 5 types of I/O system calls:

1. C CREATE

- ✓ The create() function is used to create a new empty file in C.
- ✓ We can specify the permission and the name of the file which we want to create using the create() function.
- ✓ It is defined inside **<unistd.h>** header file and the flags that are passed as arguments are defined inside **<fcntl.h>** header file.

Syntax of create() in C

```
int create(char *filename, mode_t mode);
```

Parameter:

- **filename:** name of the file which you want to create
- **mode:** indicates permissions of the new file.

Return Value:-

- ✓ return first unused file descriptor (generally 3 when first creating use in the process because 0, 1, 2 fd are reserved)
- ✓ return -1 when an error

How C create() works in OS?

- ✓ Create a new empty file on the disk.
- ✓ Create file table entry.
- ✓ Set the first unused file descriptor to point to the file table entry.
- ✓ Return file descriptor used, -1 upon failure.

2. C OPEN

The open() function in C is used to open the file for reading, writing, or both.

It is also capable of creating the file if it does not exist.

It is defined inside <**unistd.h**> header file and the flags that are passed as arguments are defined inside <**fcntl.h**> header file.

Syntax of open() in C:

```
int open (const char* Path, int flags);
```

Parameters:

Path: Path to the file which we want to open.

- ✓ Use the **absolute path** beginning with "/" when you are **not working in the same directory** as the C source file.

- ✓ Use **relative path** which is only the file name with extension, when you are **working in the same directory** as the C source file.

- ✓ **flags:** It is used to specify how you want to open the file. We can use the following flags.

Flags	Description
O_RDONLY	Opens the file in read-only mode.
O_WRONLY	Opens the file in write-only mode.
O_RDWR	Opens the file in read and write mode.
O_CREAT	Create a file if it doesn't exist.
O_EXCL	Prevent creation if it already exists.

O_ APPEND	Opens the file and places the cursor at the end of the contents.
O_ASYNC	Enable input and output control by signal.
O_CLOEXEC	Enable close-on-exec mode on the open file.
O_NONBLOCK	Disables blocking of the file opened.
O_TMPFILE	Create an unnamed temporary file at the specified path.

How C open() works in OS

- ✓ Find the existing file on the disk.
- ✓ Create file table entry.
- ✓ Set the first unused file descriptor to point to the file table entry.
- ✓ Return file descriptor used, -1 upon failure.

```c
// C program to illustrate
// open system call
#include <errno.h>
#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>
extern int errno;
int main() {
        // if file does not have in directory
        // then file foo.txt is created.
        int fd = open("foo.txt", O_RDONLY | O_CREAT);
        printf("fd = %d\n", fd);
        if (fd == -1) {
                // print which type of error have in a code
                printf("Error Number % d\n", errno);
                // print program detail "Success or failure"
                perror("Program");
        }
        return 0;
}
```

```
aman@aman-VirtualBox:~/Desktop$ ls
a.out  FCFS1.c  getchar.c  getch.c  opensystemcall.c  OS_LAB  putchar.c
aman@aman-VirtualBox:~/Desktop$ chmod +x opensystemcall.c
aman@aman-VirtualBox:~/Desktop$ cc opensystemcall.c
aman@aman-VirtualBox:~/Desktop$ ls
a.out  FCFS1.c  getchar.c  getch.c  opensystemcall.c  OS_LAB  putchar.c
aman@aman-VirtualBox:~/Desktop$ ./a.out
fd = 3
aman@aman-VirtualBox:~/Desktop$
```

3. C close

- The close() function in C tells the operating system that you are done with a file descriptor and closes the file pointed by the file descriptor.
- It is defined inside **<unistd.h>** header file.

Syntax of close() in C

```
int close(int fd);
```

Parameter

- **fd: F**ile descriptor of the file that you want to close.

Return Value

- **0** on success.
- **-1** on error.

How C close() works in the OS

How C close() works in the OS

- Destroy file table entry referenced by element fd of the file descriptor table

– As long as no other process is pointing to it!

- Set element fd of file descriptor table to **NULL**

```c
// C program to illustrate close system Call
#include <fcntl.h>
 #include <stdlib.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
        int fd1 = open("foo.txt", O_RDONLY);
        if (fd1 < 0) {
                perror("c1");
                exit(1);
        }
        printf("opened the fd = % d\n", fd1);

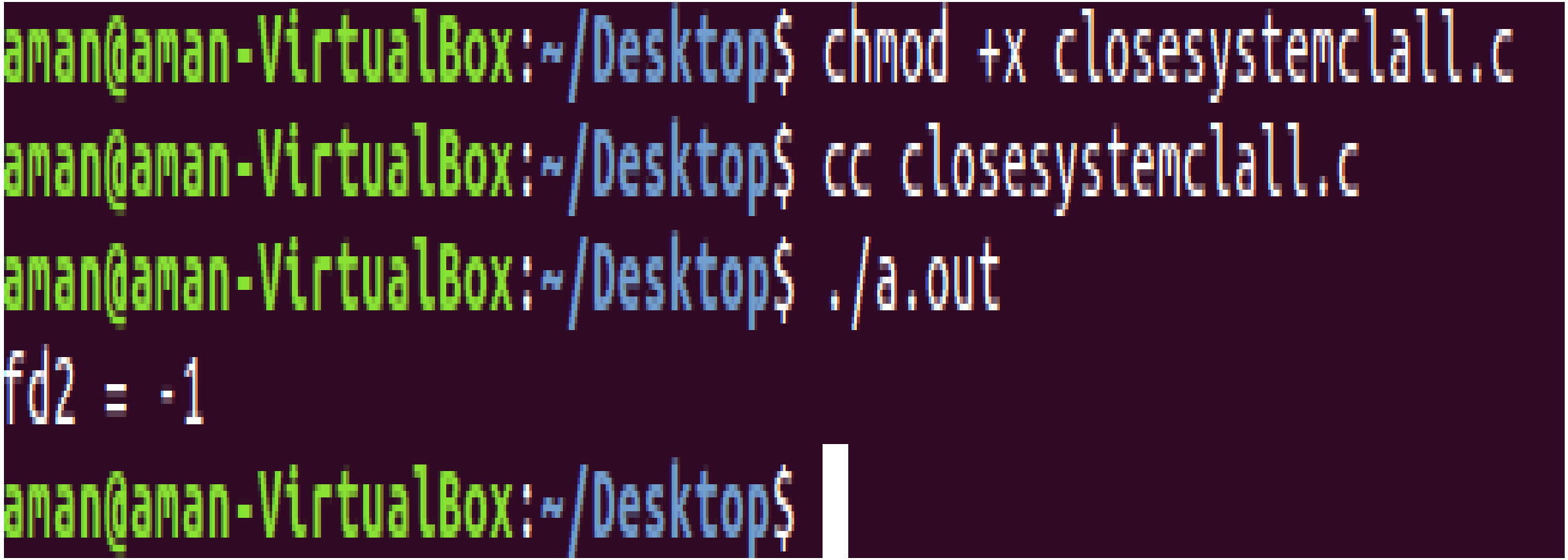
        // Using close system Call
        if (close(fd1) < 0) {
                perror("c1");
                exit(1);
        }
        printf("closed the fd.\n");
}
```

```
aman@aman-VirtualBox:~/Desktop$ chmod +x closesystemclall.c
aman@aman-VirtualBox:~/Desktop$ cc closesystemclall.c
aman@aman-VirtualBox:~/Desktop$ ./a.out
opened the fd =  3
closed the fd.
aman@aman-VirtualBox:~/Desktop$
```

```c
// C program to illustrate close system Call
#include<stdio.h>
#include<stdlib.h>
#include<fcntl.h>
#include <unistd.h>
int main()
{
        // assume that foo.txt is already created
        int fd1 = open("foo.txt", O_RDONLY, 0);
        close(fd1);

        // assume that baz.tzt is already created
        int fd2 = open("baz.txt", O_RDONLY, 0);

        printf("fd2 = % d\n", fd2);
        exit(0);
return 0;
}
```


aman@aman-VirtualBox:~/Desktop$ chmod +x closesystemclall.c
aman@aman-VirtualBox:~/Desktop$ cc closesystemclall.c
aman@aman-VirtualBox:~/Desktop$ ./a.out
fd2 = -1
aman@aman-VirtualBox:~/Desktop$

4. C Read Function

- From the file indicated by the file descriptor fd, the read() function reads the specified amount of bytes **cnt** of input into the memory area indicated by **buf**.
- A successful read() updates the access time for the file.
- The read() function is also defined inside the <unistd.h> header file.

Syntax of read() in C

```
size_t read (int fd, void* buf, size_t cnt);
```

Parameters

- **fd:** file descriptor of the file from which data is to be read.
- **buf:** buffer to read data from
- **cnt:** length of the buffer

Return Value

- return Number of bytes read on success
- return 0 on reaching the end of file
- return -1 on error
- return -1 on signal interrupt

Important Points

- **buf** needs to point to a valid memory location with a length not smaller than the specified size because of overflow.
- **fd** should be a valid file descriptor returned from open() to perform the read operation because if fd is NULL then the read should generate an error.
- **cnt** is the requested number of bytes read, while the return value is the actual number of bytes read. Also, some times read system call should read fewer bytes than cnt.

```c
// C program to illustrate
// read system Call
#include <fcntl.h>
#include <stdio.h>
#include <errno.h>
#include <stdlib.h>
#include <unistd.h>

int main()
{
        int fd, sz;
        char* c = (char*)calloc(100, sizeof(char));

        fd = open("foo.txt", O_RDONLY);
        if (fd < 0) {
                perror("r1");
                exit(1);
        }

        sz = read(fd, c, 10);
        printf("called read(% d, c, 10). returned that"
                " %d bytes were read.\n",
                fd, sz);
        c[sz] = '\0';
        printf("Those bytes are as follows: %s\n", c);

        return 0;
}
```

```
aman@aman-VirtualBox:~/Desktop$ cc readsystemcall.c
aman@aman-VirtualBox:~/Desktop$ ./a.out
called read( 3, c, 10). returned that 0 bytes were read.
Those bytes are as follows:
aman@aman-VirtualBox:~/Desktop$
```

5. C write

- Writes **cnt** bytes from **buf** to the file or socket associated with fd.
- **cnt** should not be greater than INT_MAX (defined in the limits.h header file).
- If cnt is zero, write() simply returns 0 without attempting any other action.
- The write() is also defined inside **<unistd.h>** header file.

Syntax of write() in C

size_t write (int *fd*, void* *buf*, size_t *cnt*);

Parameters

- **fd:** file descriptor
- **buf:** buffer to write data to.
- **cnt:** length of the buffer.

Return Value

- returns the number of bytes written on success.
- return 0 on reaching the End of File.
- return -1 on error.
- return -1 on signal interrupts.

Important Points about C write

- The file needs to be opened for write operations
- **buf** needs to be at least as long as specified by **cnt** because if **buf** size is less than the **cnt** then **buf** will lead to the overflow condition.
- **cnt** is the requested number of bytes to write, while the return value is the actual number of bytes written. This happens when **fd** has a less number of bytes to write than cnt.
- If write() is interrupted by a signal, the effect is one of the following:-
  - If write() has not written any data yet, it returns -1 and sets errno to EINTR.
  - If write() has successfully written some data, it returns the number of bytes it wrote before it was interrupted.

.

THE END!!!!!!